

# AFRILANG Stdlib

## std/c/graph topo

**Français.** Bibliothèque AFRILANG 'std/c/graph topo' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : topoSort, hasCycleDirected, inDegrees, outDegrees, sources, sinks, isDag.

```
'import "std/c/graph topo"' · 'use graph topo'
```

```
- 'topoSort(adj list number, n number) → list number' - 'hasCycleDirected(adj list number, n number) → bool' - 'inDegrees(adj list number, n number) → list number' - 'outDegrees(adj list number, n number) → list number' - 'sources(adj list number, n number) → list number' - 'sinks(adj list number, n number) → list number' - 'isDag(adj list number, n number) → bool'
```

**English.** AFRILANG library 'std/c/graph topo' (tier complex, category complex). Exports 7 function(s): topoSort, hasCycleDirected, inDegrees, outDegrees, sources, sinks, isDag.

```
'import "std/c/graph topo"' · 'use graph topo'
```

```
- 'topoSort(adj list number, n number) → list number' - 'hasCycleDirected(adj list number, n number) → bool' - 'inDegrees(adj list number, n number) → list number' - 'outDegrees(adj list number, n number) → list number' - 'sources(adj list number, n number) → list number' - 'sinks(adj list number, n number) → list number' - 'isDag(adj list number, n number) → bool'
```

|                       |                      |
|-----------------------|----------------------|
| Catégorie / Category  | Modules complex (c/) |
| Niveau / Tier         | complex              |
| Fonctions / Functions | 7                    |

June 16, 2026

## Contents

## Français

### 1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/graph topo' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : topoSort, hasCycleDirected, inDegrees, outDegrees, sources, sinks, isDag.

```
'import "std/c/graph topo"' · 'use graph topo'
```

```
- `topoSort(adj list number, n number) → list number`
- `hasCycleDirected(adj list number, n number) → bool`
- `inDegrees(adj list number, n number) → list number`
- `outDegrees(adj list number, n number) → list number`
- `sources(adj list number, n number) → list number`
- `sinks(adj list number, n number) → list number`
- `isDag(adj list number, n number) → bool`
```

### 2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graph topo"
use graph topo
```

Niveau : complex · Catégorie : Modules complex (c/)  
Domaines avancés – graphes, NLP, réseaux complexes.

### 3. Référence API

- topoSort(adj list number, n number) → list•  
hasCycleDirected(adj list number, n number) → bool•  
inDegrees(adj list number, n number) → list•  
outDegrees(adj list number, n number) → list•  
sources(adj list number, n number) → list•  
sinks(adj list number, n number) → list•  
isDag(adj list number, n number) → bool

### 4. Exemples d'utilisation

```
import "std/c/graph topo"
use graph topo
```

```
create result = topoSort(/* args */)
say result
```

```
create result = hasCycleDirected(/* args */)
say result
```

```
create result = inDegrees(/* args */)
say result
```

```
create result = outDegrees(/* args */)
say result
```

```
create result = sources(/* args */)
say result
```

```
create result = sinks(/* args */)
say result
```

## 5. Bonnes pratiques

- Préférez ``import "std/c/graph topo"`` en tête de fichier.
- Lancez ``afri lang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

## 6. Annexe — source

module `graph topo`

```
function topoSort(adj list number, n number) returns list number
end
```

```
function hasCycleDirected(adj list number, n number) returns bool
end
```

```
function inDegrees(adj list number, n number) returns list number
end
```

```
function outDegrees(adj list number, n number) returns list number
end
```

```
function sources(adj list number, n number) returns list number
end
```

```
function sinks(adj list number, n number) returns list number
end
```

```
function isDag(adj list number, n number) returns bool
end
end
```

## English

### 1. Overview

AFRILANG library 'std/c/graph topo' (tier complex, category complex). Exports 7 function(s): topoSort, hasCycleDirected, inDegrees, outDegrees, sources, sinks, isDag.

```
'import "std/c/graph topo"' · 'use graph topo'
```

```
- `topoSort(adj list number, n number) → list number`
- `hasCycleDirected(adj list number, n number) → bool`
- `inDegrees(adj list number, n number) → list number`
- `outDegrees(adj list number, n number) → list number`
- `sources(adj list number, n number) → list number`
- `sinks(adj list number, n number) → list number`
- `isDag(adj list number, n number) → bool`
```

### 2. Installation & import

Add the module to your program:

```
import "std/c/graph topo"
use graph topo
```

Tier: complex · Category: Complex modules (c/)  
Advanced domains – graphs, NLP, complex networks.

### 3. API reference

- topoSort(adj list number, n number) → list•  
hasCycleDirected(adj list number, n number) → bool•  
inDegrees(adj list number, n number) → list•  
outDegrees(adj list number, n number) → list•  
sources(adj list number, n number) → list•  
sinks(adj list number, n number) → list•  
isDag(adj list number, n number) → bool

### 4. Usage examples

```
import "std/c/graph topo"
use graph topo
```

```
create result = topoSort(/* args */)
say result
```

```
create result = hasCycleDirected(/* args */)
say result
```

```
create result = inDegrees(/* args */)
say result
```

```
create result = outDegrees(/* args */)
say result
```

```
create result = sources(/* args */)
say result
```

```
create result = sinks(/* args */)
say result
```

## 5. Best practices

- Prefer ``import "std/c/graphtopo"`` at the top of your file.
- Run ``afri-lang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

## 6. Appendix — source

module graphtopo

```
function topoSort(adj list number, n number) returns list number
end
```

```
function hasCycleDirected(adj list number, n number) returns bool
end
```

```
function inDegrees(adj list number, n number) returns list number
end
```

```
function outDegrees(adj list number, n number) returns list number
end
```

```
function sources(adj list number, n number) returns list number
end
```

```
function sinks(adj list number, n number) returns list number
end
```

```
function isDag(adj list number, n number) returns bool
end
end
```

## Référence rapide / Quick reference

- Import : `import "std/c/graphtopo"`
- Vérification : `afri-lang check`
- Documentation : <https://afri-lang.dev/stdlib/std/c/graphtopo/>

## Source (excerpt)

```
module graphtopo

function topoSort(adj list number, n number) returns list number
end

function hasCycleDirected(adj list number, n number) returns bool
end

function inDegrees(adj list number, n number) returns list number
end

function outDegrees(adj list number, n number) returns list number
end

function sources(adj list number, n number) returns list number
end

function sinks(adj list number, n number) returns list number
end

function isDag(adj list number, n number) returns bool
end
end
```